Tutorial: building an NBA dashboard in LibreOffice Calc

This is a hands-on walkthrough for the balldontlie NBA Calc add-in. The README is the function reference and INSTALL.md is the build guide — this document instead walks you, step by step, from "nothing installed" to a working spreadsheet that pulls live NBA data.

You'll build a small one-team dashboard (standings, latest score, roster search, a player's season stats, and a box score) using every function the add-in exposes.

---

1. What this add-in does

It adds twelve `NBA_*` worksheet functions to Calc, each backed by the balldontlie API. Type a formula like `=NBA_SCORE("2024"; "LAL")` into a cell and, a second or two later, it resolves to real NBA data — no macros, no external app, just formulas.

Two things make it behave differently from a normal function:

- **It never blocks and never errors your sheet.** A fresh request shows `#LOADING` while it fetches in the background; recalculate (**F9**) a moment later and the real value appears.
- **Table-shaped functions "spill" only via array entry.** Functions like `NBA_TEAMS` or `NBA_STANDINGS` return multiple rows/columns. LibreOffice has no dynamic spill (unlike Excel 365), so you select the output range first and confirm with **Ctrl+Shift+Enter**.

Keep those two behaviors in mind — most "it's not working" moments are one of these two.

2. Prerequisites

- LibreOffice Calc (any recent version).
- A free balldontlie API key: sign up at https://www.balldontlie.io/. A key is required on every request as of July 2026; without one every `NBA_*` cell shows `#NO_API_KEY`.
- The `.oxt` extension file itself — either downloaded prebuilt, or built from source (see step 3).

3. Install the extension

Easiest path — download the prebuilt release:

```
mkdir -p ~/.config/libreoffice-nba
echo 'apiKey=your_key' > ~/.config/libreoffice-nba/balldontlie.properties
"$LO_HOME/program/unopkg" add NBA.oxt
```

(`NBA.oxt` from the latest release.) You can also just double-click the `.oxt` file to open LibreOffice's Extension

Manager and install it that way.

Building it yourself is only necessary if you want to modify the Java source. It requires a JDK 8+ and the LibreOffice SDK; the full procedure (with platform-specific notes for Windows/Linux/macOS) is in INSTALL.md. The short version:

```
export JAVA_HOME=~/jdks/jdk8u<version>
export LO_HOME=~/libreoffice26.2
./build.sh
"$LO_HOME/program/unopkg" add --force build/NBA.oxt
```

Either way, **restart LibreOffice** after installing.

4. Configure your API key

You have four ways to supply the key (full priority order in the README); the properties file is the most convenient because it doesn't depend on how you launch LibreOffice:

```
mkdir -p ~/.config/libreoffice-nba
cat > ~/.config/libreoffice-nba/balldontlie.properties <<'EOF'
apiKey=your_real_key_here
EOF
```

Restart LibreOffice so it picks up the file. If you'd rather not touch config files, you can instead type the key into a cell (e.g. `B1`) and pass `$B$1` as the trailing `api_key` argument on every formula — useful for demo workbooks you plan to share, since the key then travels with the sheet only if you choose to fill that cell in.

5. Sanity check: your first formula

Open a new Calc document and type:

```
=NBA_TEAMID("Lakers")
```

Press Enter. You'll likely see `#LOADING` for a second — that's the background fetch kicking off. Press **F9** (recalculate) after a couple of seconds; the cell should resolve to a numeric team id (e.g. `14`).

If instead you see: - `#NAME?` → the extension isn't registered. Run `unopkg list` and confirm `com.example.nba` is present; reinstall if not. - `#NO_API_KEY` → the key didn't resolve. Recheck step 4, and make sure you restarted LibreOffice after creating the properties file. - `#ERR` → call `=NBA_LASTERROR()` in an empty cell for the detail message.

6. Build a one-team dashboard

We'll build a small dashboard around one team, e.g. the Boston Celtics. Lay out the following in a blank sheet. (`test/nba_demo.ods` in the repo is a completed, real-data example of exactly this, built around Boston's 2023-24 championship season — open it side by side if you want to compare.)

6a. Resolve the team id once

```
A1: Team name        B1: Celtics
A2: Team id           B2: =NBA_TEAMID(B1)
```

`B2` resolves to a numeric id. Several other functions (`NBA_SCORE`, `NBA_TEAMRANK`) also accept the plain abbreviation/name directly and look this up internally — but pulling it into its own cell like this means you only pay the lookup once, and you can reuse `B2` anywhere a numeric team id is required (like `NBA_GAMES`'s optional team filter).

6b. A full team roster of teams (array formula)

```
A4: =NBA_TEAMS()
```

Select a range large enough for the result — say `A4:G34` (30 teams × 7 columns: id, abbreviation, city, conference, division, full_name, name) — type the formula, then confirm with **Ctrl+Shift+Enter** instead of plain Enter. LibreOffice will fill the whole block. If you only enter it in a single cell, you'll just see the first row.

6c. Latest score

```
A6: Latest score      B6: =NBA_SCORE("2024"; "BOS")
```

Resolves to a formatted string like `"2024-06-17 BOS 106 - 88 DAL (W)"` — the most recent game in that season for that team.

6d. Standings and conference rank

```
A7: Conference rank   B7: =NBA_TEAMRANK("2024"; "BOS")
A9: Standings (East)  A10: =NBA_STANDINGS("2024"; "East")
```

`A10` is another array formula — select several rows/columns below it (conference, division, team, wins, losses, win_pct, conference_rank) and enter with Ctrl+Shift+Enter.

> Note: `/standings` and `/stats` are **paid-tier** balldontlie endpoints. On a free key, `NBA_STANDINGS`, `NBA_TEAMRANK`, `NBA_PLAYERSTAT`, and `NBA_BOXSCORE` will show `#ERR` (`NBA_LASTERROR()` → `HTTP 401: Unauthorized`) until you upgrade. `NBA_TEAMID`, `NBA_TEAMS`, `NBA_PLAYERID`, `NBA_PLAYERSEARCH`, `NBA_GAMES`, and `NBA_SCORE` all work on the free tier — the rest of this tutorial's steps below need a paid key to actually resolve, but will still enter correctly as formulas.

6e. Find a player and pull their season stats

```
A12: Player search  A13: =NBA_PLAYERSEARCH("James")
```

Array formula again — spills id/first_name/last_name/position/team, up to 100 matches. Once you spot the player you want (e.g. LeBron James):

```
A15: Player id       B15: =NBA_PLAYERID("LeBron James")
```

```
A16: Season PPG      B16: =NBA_PLAYERSTAT("2024"; B15; "pts")
```

`NBA_PLAYERSTAT` averages the requested `stat_key` (`pts`, `reb`, `ast`, `stl`, `blk`, `min`, `fg_pct`, `fg3_pct`, `ft_pct`, …) across that player's games in the season — a simple mean, not attempt-weighted for percentage fields.

6f. A game's full box score

You'll need a numeric `game_id` — the demo workbook has one baked in, or you can spot one from a `NBA_GAMES` result (see below). Given one:

```
A18: Box score  A19: =NBA_BOXSCORE(15908)
```

Array formula — spills one row per player: player, team, min, pts, reb, ast, stl, blk, fg_pct, fg3_pct, ft_pct.

6g. All games on a given day, filtered to one team

```
A21: Games   A22: =NBA_GAMES("2024-01-15")
```

or, filtered client-side to just Boston's games that season, pass the numeric id from step 6a:

```
A24: BOS games (season)  A25: =NBA_GAMES("2024"; B2)
```

Both are array formulas: date, home_team, home_score, away_team, away_score, status.

6h. Diagnostics

Two housekeeping formulas, handy while you're building:

```
=NBA_LASTERROR()   -> "" normally, or the detail behind the most recent #ERR
=NBA_CACHECLEAR()  -> clears the shared cache; returns the count cleared
```

`NBA_CACHECLEAR()` is useful once you've fixed a bad key or want to force a fresh pull instead of waiting out the TTL (see step 8).

7. Recalculating

Because everything resolves asynchronously against a background cache, `#LOADING` cells need a nudge to update once the fetch completes:

- **F9** recalculates the active sheet.
- **Ctrl+Shift+F9** force-recalculates everything — use this if F9 doesn't seem to clear a `#LOADING`, since some dependent formulas need a second pass after the one that triggered the fetch resolves.

Expect the very first uncached formula in a session to take a couple of seconds; if you enter several *different* fresh requests back to back, each one can take up to ~13 seconds due to the free-tier rate limit (see below).

8. Understanding caching, errors, and rate limits

Cell shows	Meaning
`#LOADING`	First request for this exact query. Recalculate shortly.
`#NO_API_KEY`	No key resolved — see step 4.
`#NOT_FOUND`	Reached the API, nothing matched (typo'd team, empty search, team absent from that season's standings).
`#ERR`	Persistent fetch failure. Check `NBA_LASTERROR()`; retried automatically ~15s later.

Responses are cached in memory for the life of the LibreOffice session, TTL depending on how often the underlying data changes: teams 24h, standings 1h, games/scores/box scores 5min, player search 6h. You keep seeing the last-known-good value while a stale entry refreshes silently in the background — call `NBA_CACHECLEAR()` to force a clean pull.

The free API tier allows **5 requests/minute**. The add-in throttles itself to match (~13s between outgoing requests) and retries 429/5xx with backoff, so normal use rarely hits the limit — but a burst of several *different* fresh formulas will visibly queue up.

9. Using a per-cell API key instead of the environment

Every data function takes an optional trailing `api_key` argument, which overrides whatever the environment resolves to for that one call:

```
=NBA_TEAMID("Lakers"; $B$1)
=NBA_SCORE("2024"; "LAL"; ; $B$1)     <- note the empty middle arg for NBA_SCORE's
optional `nth`
```

This is how the demo workbook (`test/nba_demo.ods`) stays shareable: it has a placeholder key cell rather than a real key baked into formulas. Prefer a cell reference over typing the key literally into a formula, so it isn't spelled out (and duplicated) in every cell that uses it.

10. Where to go next

- **Full function reference** (every signature, every return shape): the table at the top of README.md.
- **Behavior details** worth knowing before you build something serious — name-matching quirks for `NBA_PLAYERID`/`NBA_PLAYERSEARCH`, why `NBA_GAMES`'s team filter wants a numeric id, `CompatibilityName` XLS/XLSX round-tripping: see "Behavior notes" in the README.
- **Building from source / troubleshooting the build**: INSTALL.md.
- **A finished example**: `test/nba_demo.ods`, regenerate with `tools/build_demo.py` against a headless LibreOffice instance.